

DANESHVAR AMROLLAHI

✉ daneshvar@cs.stanford.edu  cs.stanford.edu/~daneshva  github.com/daneshvar-amrollahi
389 Jane Stanford Way, CoDa #W310, Stanford, CA 94305, United States

EDUCATION

- **Stanford University** 2024/01 - Present
PhD in Computer Science Advisor: Clark Barrett
- **Stanford University** 2024/01 - 2025/06
MSc in Computer Science
- **University of Tehran** 2018/09 - 2023/02
BSc in Computer Engineering (Software) GPA: 18.02/20.00

PUBLICATIONS

- D. Amrollahi, M. Karimi (2026). **Type-Checking Bach: A Programming Languages Approach to Tonal Music.** *Under review at Conference on Programming Language Design and Implementation (PLDI 2026).*
- C. Sun, Y. Sun, D. Amrollahi, E. Zhang, S. Lahiri, S. Lu, D. Dill, C. Barrett (2025). **VeriStruct: AI-assisted Automated Verification of Data-Structure Modules in Verus.** *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2026)*
- D. Amrollahi, M. Preiner, A. Niemetz, A. Reynolds, M. Charikar, C. Tinelli, C. Barrett (2025). **Towards SMT Solver Stability via Input Normalization.** *Formal Methods in Computer-Aided Design (FMCAD 2025).*
- P. Hozzová, D. Amrollahi, M. Hajdu, L. Kovács, A. Voronkov, E.M. Wagner (2024). **Synthesis of Recursive Programs in Saturation.** *International Joint Conference on Automated Reasoning (IJCAR 2024).*
- D. Amrollahi, E. Bartocci, G. Kenison, L. Kovács, M. Moosbrugger, M. Stankovič (2024). **(Un)Solvable Loop Analysis.** *Formal Methods in System Design (FMSD).*
- D. Amrollahi, H. Hojjat, P. Rümmer (2024). **An Encoding for CLP Problems in SMT-LIB.** *Workshop on Horn Clauses for Verification and Synthesis (HCVS 2023).*
- D. Amrollahi, E. Bartocci, G. Kenison, L. Kovács, M. Moosbrugger, M. Stankovič (2022). **Solving Invariant Generation for Unsolvable Loops.** *International Static Analysis Symposium (SAS 2022).* **Awarded the Radhia Cousot Young Researcher Best Paper Award.**
- A. Humenberger, D. Amrollahi, N. Bjørner, L. Kovács (2022). **Algebra-Based Reasoning for Loop Synthesis.** *Formal Aspects of Computing (FAC).*

INDUSTRY EXPERIENCE

- **Applied Science Intern at Amazon Web Services (AWS)** Santa Clara, United States
Automated Reasoning Group 2025/06 – 2025/09
Built a tool from scratch for validating SMT-LIB arithmetic queries and models: integrated a Rust parser to construct Lean objects, developed a Rust–Lean FFI pipeline, encoded SMT terms and semantics in **Lean**, and formally proved correctness of optimized evaluation against reference semantics.

RESEARCH EXPERIENCE

- **PhD Student at Center for Automated Reasoning, Stanford University** Stanford, United States
Under Prof. Clark Barrett 2024/01 – Present
 1. Working on **auto-formalization** of rules of the road using **LLMs**, translating natural-language descriptions into formal logic representations for **safety validation** in self-driving trucks.
 2. Working on using **LLMs** to learn programmatic features from **verification** benchmarks to tune solver/verifier heuristics, bridging neural and **formal reasoning** for more adaptive and interpretable verification.
 3. Implemented SMT query normalization in C++, enhancing performance **stability** of **cvc5** and **Z3** across query mutations for all SMT-LIB theories, including industrial **program verification** benchmarks.

- **Research Intern at Dependable Systems Lab, EPFL**

Lausanne, Switzerland

Under Prof. George Candea

2022/07 – 2022/08

Extended the KLEE **symbolic execution** engine to generate and track **quantified formulas** in **Z3**, enabling **loop summarization** to mitigate **path explosion** and improve scalability in analyzing **network software**. Involved substantial C++ **systems programming** and **SMT solver** integration.

- **Research Intern at Automated Program Reasoning Group, TU Wien**

Vienna, Austria

Under Prof. Laura Kovács and Prof. Ezio Bartocci

2021/07 - 2022/02 + 2023/05 - 2023/12

1. Extended state-of-the-art techniques for generating **polynomial loop invariants**, enabling support for a broader class of loops. Implemented in Polar, a **static analysis** tool for **reasoning** about **probabilistic programs** using **symbolic** approaches.
2. Implemented a recursive **program synthesis** framework based on mathematical induction over algebraic datatypes, into Vampire, a state-of-the-art first-order **theorem prover** written in C++.

HONORS AND AWARDS

- **Radhia Cousot Young Researcher Best Paper Award**

2022

29th Static Analysis Symposium (SAS 2022)

- **Stanford School of Engineering (SoE) Fellowship**

2024

Awarded a \$12900/quarter stipend and full tuition coverage for one year

- **Ranked 8th in ACM-ICPC (International Collegiate Programming Contest)**

2020

West Asia Region, Tehran site

- **Summer@EPFL Fellowship**

2022

Ranked top 1.5% among 4000 applicants and awarded a 1600 CHF/month fellowship over summer

PROJECTS

- **cvc5** — github.com/cvc5/cvc5

C++

Implemented a normalization pre-processing pass in this industrial-strength SMT solver for achieving performance stability across query mutations.

- **KLEE** — github.com/bolt-perf-contracts/klee/pull/9

C++, Z3

Integrated Z3's quantified reasoning into KLEE's symbolic execution engine to summarize loops and mitigate path explosion, improving scalability on complex programs involving heavy string operations.

- **ARM Processor Pipeline** — github.com/daneshvar-amrollahi/ARM

Verilog, ModelSim, Quartus

Implemented a comprehensive 5-stage pipelined ARM processor with data forwarding, hazard detection, cache memory, and SRAM controller.

- **Feed-forward Neural Network** — github.com/daneshvar-amrollahi/FNN-Verilog

Verilog, ModelSim

A full hardware implementation of a feedforward neural network designed for digit classification using Multiply-Accumulate (MAC) units, with a 4-clock cycle inference latency.

- **Vampire** — github.com/vprover/vampire

C++

Implemented a framework within this theorem prover for synthesizing verified recursive programs.

- **Polar** — github.com/probing-lab/polar

Python, SymPy

Implemented a polynomial loop-invariant synthesizer for (probabilistic) unsolvable loops.

- **Koloocheh** — github.com/daneshvar-amrollahi/Koloocheh

Python, gRPC

A peer-to-peer file-sharing system employs the flooding algorithm for search operations and ensures a small graph diameter by leveraging random graph properties.

- **xv6** — github.com/daneshvar-amrollahi/os-lab-xv6

C

Extended xv6 operating system kernel with several extra features such as various new system calls, multilevel queue scheduling, and synchronization.

- **FTP Server** — github.com/daneshvar-amrollahi/FTP-Server

C++, Posix Threads

A multithreaded client-server implementation of an FTP Server using sockets for communication.

SKILLS

- **Programming Languages:**
 - Experienced in C, C++, Python.
 - Familiar with Scala, Rust, Go, Bash, Verilog, SystemVerilog.
- **Tools:** cvc5, Z3, Lean, KLEE, L^AT_EX.
- **Academic (Sub)review:** ISSAC 2024, TACAS 2026, Journal of Symbolic Computation.